

所谓回文数，指的是数字正序排列和逆序排列都是同一数值的数，比如，数字1221 按正序和逆序排列都是1221，因此1221 是回文数，而数字1234按逆序排列是4321，4321 与1234 不是同1 个数，因此1234 不是回文数。请编写程序，获取用户输入的4 位数字，并判断用户输入的数字是否是回文数。

输出结果如下图所示（注：标点符号为全角符号）

示例1

请输入长度为4位的数字：1221
1221是一个回文数！

示例2

请输入长度为4位的数字：1456
1456不是一个回文数！

示例3

请输入长度为4位的数字：56
错误，输入的数字长度不为4。

示例输入：1221

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入长度为4位的数字：");
        String s = sc.next();
        if(s.length() != 4){
            System.out.println("错误，输入的数字长度不为4。");
        }else{
            if(s.charAt(0)==s.charAt(3) && s.charAt(1)==s.charAt(2)){
                System.out.println(s+"是一个回文数！");
            }else{
                System.out.println(s+"不是一个回文数！");
            }
        }
    }
}
```

假定小鸡1元3只，公鸡3元1只，母鸡5元1只。请输入购鸡钱数用于购买100只鸡，请编写程序列出所有的购鸡方案。

输出结果如下图所示（注：输入的钱数为整数，标点符号为全角符号）

示例1

请输入购鸡的钱数：100
100钱买百鸡的方案如下
母鸡：0只，公鸡：25只，小鸡：75只
母鸡：4只，公鸡：18只，小鸡：78只
母鸡：8只，公鸡：11只，小鸡：81只
母鸡：12只，公鸡：4只，小鸡：84只

示例2

请输入购鸡的钱数：50
50钱买百鸡的方案如下
母鸡：3只，公鸡：1只，小鸡：96只

示例3

请输入购鸡的钱数：145
145钱买百鸡的方案如下
没有可执行的方案！

示例输入：100

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入购鸡的钱数：");
        int money = sc.nextInt();
        System.out.println(money + "钱买百鸡的方案如下");

        boolean found = false;

        for (int h = 0; h <= 100; h++) {
            for (int r = 0; r <= 100 - h; r++) {
                int c = 100 - h - r;

                if (c >= 0 && c % 3 == 0 && 5 * h + 3 * r + c / 3 == money) {
                    System.out.println("母鸡：" + h + "只，公鸡：" + r + "只，小鸡：" + c + "只");
                    found = true;
                }
            }
        }

        if (!found) {
            System.out.println("没有可执行的方案！");
        }
    }
}
```

已知某快递点提供华东地区、华南地区、华北地区的寄件服务，其中华北地区编号为1、华东地区编号为2、华南地区编号为3，该快递点寄件价目表具体如表1 所示。

表1 寄件价目表

地区编号	首重 (<=2kg)	续重 (元/kg)
1 (华北地区)	12 元	2 元
2 (华东地区)	13 元	3 元
3 (华南地区)	14 元	3 元

假如用户在华北地区邮寄4kg 商品，快递收费公式为：首重+续重*2，即12+（4-2）*2，共计16 元。

本任务要求编写代码，用户在控制台输入寄件地区编号和快递重量，实现根据表1 的价格计算快递费用的程序。

输出结果如下图所示（注：输入的数值为整数，标点符号为全角符号）

示例1

请输入寄件的地区编号：4
输入的寄件的地区编号有误！

示例2

请输入寄件的地区编号：2
请输入快递重量（kg）：4
本次快递费为：19元

示例输入：2 5

参考答案（参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入寄件的地区编号：");
        int area = sc.nextInt();
        if (area < 1 || area > 3) {
            System.out.println("输入的寄件的地区编号有误！");
        } else {
            System.out.print("请输入快递重量（kg）：");
            int weight = sc.nextInt();
            int money = 0;
            if (area == 1) {
                money = 12;
                if (weight > 2) {
                    money += (weight - 2) * 2;
                }
            } else if (area == 2) {
                money = 13;
                if (weight > 2) {
                    money += (weight - 2) * 3;
                }
            } else if (area == 3) {
                money = 14;
                if (weight > 2) {
                    money += (weight - 2) * 3;
                }
            }
            System.out.println("本次快递费为：" + money + "元");
        }
    }
}
```

利用switch语句和输入流类Scanner，实现简单的四则运算。

提示： (1) Scanner类的nextDouble()方法用于输入双精度浮点数；next()方法用于输入字符串。

(2) String类的charAt()方法可以获取字符串中指定索引的字符。

运行结果示例1:

```
请输入第一个操作数 (例如: 12) : 12
请输入四则运算符 (例如: +、-、*、/) : +
请输入第二个操作数 (例如: 8) : 8
12.0+8.0=20.0
```

运行结果示例2:

```
请输入第一个操作数 (例如: 12) : 12
请输入四则运算符 (例如: +、-、*、/) : -
请输入第二个操作数 (例如: 8) : 8
12.0-8.0=4.0
```

运行结果示例3:

```
请输入第一个操作数 (例如: 12) : 12
请输入四则运算符 (例如: +、-、*、/) : *
请输入第二个操作数 (例如: 8) : 2
12.0*2.0=24.0
```

运行结果示例4:

```
请输入第一个操作数 (例如: 12) : 12
请输入四则运算符 (例如: +、-、*、/) : /
请输入第二个操作数 (例如: 8) : 2
12.0/2.0=6.0
```

示例输入: 12 + 8

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入第一个操作数 (例如: 12) : ");
        double num1 = sc.nextDouble();
        System.out.print("请输入四则运算符 (例如: +、-、*、/) : ");
        String opStr = sc.next();
        char op = opStr.charAt(0);
        System.out.print("请输入第二个操作数 (例如: 8) : ");
        double num2 = sc.nextDouble();
        double res = 0;
        switch (op) {
            case '+':
                res = num1 + num2;
                break;
            case '-':
                res = num1 - num2;
                break;
            case '*':
                res = num1 * num2;
                break;
            case '/':
                res = num1 / num2;
                break;
            default:
                System.out.println("运算符错误");
                return;
        }
        System.out.println(num1 + (" " + op) + num2 + "=" + res);
    }
}
```

计算 $1-3+5-7+\dots-99+101$ 的值。

运行结果如下：

1-3+5-7+...-99+101的值是： 51

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int sum = 0, flag = 1;
        for(int i = 1; i <= 101; i += 2) {
            sum += flag * i;
            flag = -flag;
        }
        System.out.println("1-3+5-7+...-99+101的值是：" + sum);
    }
}
```

求数列 $2/1, 3/2, 5/3, 8/5, 13/8, 21/13, \dots$ 的前 n 项的和。

运行结果示例1:

输入n: 5
前5项的和是: 8.391666666666667

运行结果示例2:

输入n: 10
前10项的和是: 16.479905306194137

示例输入: 5

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("输入n: ");
        int n = sc.nextInt();

        double sum = 0.0;
        double a = 2;
        double b = 1;

        for (int i = 0; i < n; i++) {
            sum += a / b;
            double temp = a;
            a = a + b;
            b = temp;
        }

        System.out.println("前" + n + "项的和是: " + sum);
    }
}
```

输入一个字符串，统计该字符串中有多少个数字字符。

提示：字符串的charAt()方法，可以获取指定索引位置的字符。

运行结果示例：

```
请输入一个字符串：aB23c5d
aB23c5d中数字字符的数量是：3
```

示例输入：abc123def45

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入一个字符串：");
        String str = sc.nextLine();
        int count = 0;
        for(int i = 0; i < str.length(); i++){
            char c = str.charAt(i);
            if(c >= '0' && c <= '9'){
                count++;
            }
        }
        System.out.println(str + "中数字字符的数量是：" + count);
    }
}
```

输入一个整数n，计算从1到n（包括n）中所有能够被2或者被3整除的数之和。

运行结果示例1:

请输入一个整数n: 10
1到10中所有能够被2或者被3整除的数之和是: 42

运行结果示例2:

请输入一个整数n: 20
1到20中所有能够被2或者被3整除的数之和是: 137

示例输入: 10

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入一个整数n: ");
        int n = sc.nextInt();
        int sum = 0;
        for(int i = 1; i <= n; i++){
            if(i % 2 == 0 || i % 3 == 0){
                sum += i;
            }
        }
        System.out.println("1到" + n + "中所有能够被2或者被3整除的数之和是: " + sum);
    }
}
```

求出从键盘上输入的n个正整数中的最大值。

运行结果示例1:

```
请输入正整数的个数n: 3
请输入3个正整数:
2
3
1
最大值为: 3
```

运行结果示例2:

```
请输入正整数的个数n: 4
请输入4个正整数:
100
5
40
90
最大值为: 100
```

示例输入: 5 / 3 7 2 9 4

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入正整数的个数n: ");
        int n = sc.nextInt();
        System.out.println("请输入" + n + "个正整数: ");
        int max = 0;
        for(int i = 0; i < n; i++){
            int num = sc.nextInt();
            if(num > max){
                max = num;
            }
        }
        System.out.println("最大值为: " + max);
    }
}
```

求出 $m \sim n$ 以内（包括 m 和 n ）的所有整数之和，其中 m 和 n 由用户输入($m \leq n$)，程序执行后输出求和结果。

运行结果示例1:

```
请输入整数m (m <= n) : 5
请输入整数n (m <= n) : 10
5~10的所有整数之和为: 45
```

运行结果示例2:

```
请输入整数m (m <= n) : 10
请输入整数n (m <= n) : 5
m应该小于等于n
```

示例输入: 1 100

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入整数m (m <= n) : ");
        int m = sc.nextInt();
        System.out.print("请输入整数n (m <= n) : ");
        int n = sc.nextInt();

        if(m > n){
            System.out.println("m应该小于等于n");
        }else{
            int sum = 0;
            for(int i = m; i <= n; i++){
                sum += i;
            }
            System.out.println(m + "~" + n + "的所有整数之和为: " + sum);
        }
    }
}
```

计算序列 $S_n = a + aa + aaa + \dots + \underbrace{aa \dots a}_n$ 的和，其中 a 是一个数字，序列中有 n 项，每一项由 a 重复构成。 a 和 n 的值均由键盘输入，输出序列之和 S_n 。

运行结果示例1:

```
请输入数字a: 2
请输入项数n: 3
序列的和Sn为: 246
```

运行结果示例2:

```
请输入数字a: 1
请输入项数n: 5
序列的和Sn为: 12345
```

示例输入: 2 3

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入数字a: ");
        int a = sc.nextInt();
        System.out.print("请输入项数n: ");
        int n = sc.nextInt();

        long sum = 0;
        long item = 0;
        for (int i = 0; i < n; i++) {
            item = item * 10 + a;
            sum += item;
        }
        System.out.println("序列的和Sn为: " + sum);
        sc.close();
    }
}
```

使用循环或递归方法，计算 $n!$

(1) 定义静态方法fact()，接受参数n，计算 $n!$

(2) 在main方法中，调用fact()方法计算阶乘，n由键盘输入，输出返回的结果。

运行结果示例1:

```
请输入n: 3
3! = 6
```

运行结果示例2:

```
请输入n: 5
5! = 120
```

示例输入: 5

参考答案 (参考实现，已通过本地测试用例，仅供对照)

```
import java.util.Scanner;

public class Main {

    public static long fact(int n) {
        if (n == 0 || n == 1) {
            return 1;
        }
        return n * fact(n - 1);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入n: ");
        int n = sc.nextInt();
        long res = fact(n);
        System.out.println(n + "! = " + res);
        sc.close();
    }
}
```

输入一组数据，当输入0时输入结束，统计输入数据的个数及输入数据的平均值（小数点后保留1位）。

提示：在System.out.printf()中，%.1f 表示将浮点数格式化为小数点后保留1位小数的字符串。

运行结果示例1:

请输入一组数据（输入0结束）：

1
2
3
4
0

输入数据的个数：4
输入数据的平均值：2.5

运行结果示例2:

请输入一组数据（输入0结束）：

3
4
10
0

输入数据的个数：3
输入数据的平均值：5.7

示例输入：85 92 78 90 0

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int count = 0;
        double sum = 0;

        System.out.println("请输入一组数据（输入0结束）：");

        double num = input.nextDouble();

        while (num != 0) {
            sum += num;
            count++;
            num = input.nextDouble();
        }

        System.out.println("输入数据的个数：" + count);
        System.out.printf("输入数据的平均值：%.1f", sum / count);
    }
}
```

从键盘输入3个整数a,b,c的值，要求按从大到小的顺序输出。

运行结果示例1:

```
1
2
3
3, 2, 1
```

运行结果示例2:

```
5
2
4
5, 4, 2
```

示例输入: 3 1 2

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        int c = sc.nextInt();
        int temp;

        if (a < b) {
            temp = a;
            a = b;
            b = temp;
        }
        if (a < c) {
            temp = a;
            a = c;
            c = temp;
        }

        if (b < c) {
            temp = b;
            b = c;
            c = temp;
        }
        System.out.println(a + "," + b + "," + c);
        sc.close();
    }
}
```

编写程序计算下面分段函数的值：

$$f(x) = x^3 + 5 \quad (x \leq 0)$$

$$f(x) = x^2 - x + 3 \quad (0 < x \leq 20)$$

$$f(x) = 7x + 2 \quad (x > 20)$$

运行结果示例1：

```
请输入x的值：
-2
f(x)的值为： -3.0
```

运行结果示例2：

```
请输入x的值：
2
f(x)的值为： 5.0
```

运行结果示例3：

```
请输入x的值：
30
f(x)的值为： 212.0
```

示例输入： 2

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("请输入x的值：");
        double x = sc.nextDouble();
        double f;
        if (x <= 0) {
            f = x * x * x + 5;
        } else if (x <= 20) {
            f = x * x - x + 3;
        } else {
            f = 7 * x + 2;
        }
        System.out.println("f(x)的值为：" + f);
        sc.close();
    }
}
```

输入一正整数k，求出它是几位数，并按逆序输出各位数字。

提示：String.valueOf()用于将int转换为String。

运行结果示例1:

```
请输入一个正整数: 123
123是一个3位数, 逆序输出为: 321
```

运行结果示例2:

```
请输入一个正整数: 1234
1234是一个4位数, 逆序输出为: 4321
```

示例输入: 12345

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入一个正整数: ");
        int k = sc.nextInt();

        String s = String.valueOf(k);
        int len = s.length();

        String reverse = "";
        for (int i = len - 1; i >= 0; i--) {
            reverse += s.charAt(i);
        }
        System.out.println(k + "是一个" + len + "位数, 逆序输出为: " + reverse);
        sc.close();
    }
}
```

如果一个整数的各位数字的立方和等于该数本身，这个整数就是水仙花数，如 $153=1^3+5^3+3^3$ 。输入一个3位数，判断其是否为水仙花数，要求如下：

- (1) 定义静态方法is()，判断输入参数的各位数字之立方和是否等于其自身。
- (2) 在main方法中，输入一个3位数，调用is()方法，判断其是否为3位水仙花数。

运行结果示例1：

```
输入一个三位整数：153
153是否为水仙花数？ true
```

运行结果示例2：

```
输入一个三位整数：350
350是否为水仙花数？ false
```

示例输入：153

参考答案 (参考实现，已通过本地测试用例，仅供对照)

```
import java.util.Scanner;
public class Main {

    public static boolean is(int n) {
        int a = n / 100;
        int b = n / 10 % 10;
        int c = n % 10;
        int sum = a * a * a + b * b * b + c * c * c;
        return sum == n;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("输入一个三位整数：");
        int num = sc.nextInt();
        boolean res = is(num);
        System.out.println(num + "是否为水仙花数？" + res);
        sc.close();
    }
}
```

有1、2、3、4四个数字，能组成多少个互不相同且无重复数字的三位数？都是多少？

运行结果如下：

```
123
124
132
134
142
143
213
214
231
234
241
243
312
314
321
324
341
342
412
413
421
423
431
432
共有24种组合
```

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
public class Main {
    public static void main(String[] args) {
        int count = 0;
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 4; j++) {
                for (int k = 1; k <= 4; k++) {

                    if (i != j && j != k && i != k) {
                        int num = i * 100 + j * 10 + k;
                        System.out.println(num);
                        count++;
                    }
                }
            }
        }
        System.out.println("共有" + count + "种组合");
    }
}
```

将数组{8, 2, 6, 5, 9, 4, 1, 3}的值按逆序重新存放并输出。

运行结果如下：

3 1 4 9 5 6 2 8

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {
            8,
            2,
            6,
            5,
            9,
            4,
            1,
            3
        };
        int len = arr.length;

        for (int i = 0; i < len / 2; i++) {
            int temp = arr[i];
            arr[i] = arr[len - 1 - i];
            arr[len - 1 - i] = temp;
        }

        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

从键盘输入5个数存入一个数组，然后输出该数组中的最小值。

运行结果示例1:

```
请输入5个整数:
10
5
8
2
4
数组中的最小值是: 2
```

运行结果示例2:

```
请输入5个整数:
5
1
0
-10
100
数组中的最小值是: -10
```

示例输入: 5 3 8 1 9

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        System.out.println("请输入5个整数: ");
        for (int i = 0; i < 5; i++) {
            arr[i] = sc.nextInt();
        }
        int min = arr[0];
        for (int i = 1; i < 5; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        System.out.println("数组中的最小值是: " + min);
        sc.close();
    }
}
```

在有序数组{1, 3, 5, 7, 9, 11, 13, 15}中，查找某个元素（由键盘输入），若查找成功返回该元素下标，否则返回-1。

运行结果示例1:

请输入要查找的数字: 9
查找结果: 4

运行结果示例2:

请输入要查找的数字: 10
查找结果: -1

示例输入: 7

参考答案 (参考实现, 已通过本地测试用例, 仅供对照)

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 3, 5, 7, 9, 11, 13, 15};
        Scanner sc = new Scanner(System.in);
        System.out.print("请输入要查找的数字: ");
        int target = sc.nextInt();

        int left = 0;
        int right = arr.length - 1;
        int index = -1;
        //二分查找
        while(left <= right){
            int mid = (left + right)/2;
            if(arr[mid] == target){
                index = mid;
                break;
            }else if(arr[mid] < target){
                left = mid + 1;
            }else{
                right = mid -1;
            }
        }
        System.out.println("查找结果: " + index);
        sc.close();
    }
}
```

练习：猜数游戏。有一个数列：8，4，2，1，23，344，12。

- (1) 用循环输出数列的所有值。
- (2) 求数列中所有数值的和。
- (3) 求最大值及其所在的位置。
- (4) 从键盘输入一个整数，判断数列中是否包含此数。

输出约定：第一行输出数列，第二行输出总和，第三行输出最大值、下标和位置，第四行输出查找结果。数组下标从0开始，位置从1开始；344的下标为5，位置为6。

示例输入23时，输出：

8 4 2 1 23 344 12

数列的和：394

最大值：344，下标：5，位置：6

数列中包含23

输入7时，最后一行输出：数列中不包含7。

示例输入：23

参考答案 （参考实现，已通过本地测试用例，仅供对照）

```
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        int[] arr = {8, 4, 2, 1, 23, 344, 12};
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + (i == arr.length - 1 ? "\n" : " "));
        }
        int sum = 0;
        for (int i = 0; i < arr.length; i++) {
            sum += arr[i];
        }
        System.out.println("数列的和: " + sum);
        int maxpos = 0, max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > max) {
                max = arr[i];
                maxpos = i;
            }
        }
        System.out.println("最大值: " + max + ", 下标: " + maxpos
            + ", 位置: " + (maxpos + 1));
        Scanner sc = new Scanner(System.in);
        int target = sc.nextInt();
        boolean found = false;
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                found = true;
                break;
            }
        }
        System.out.println("数列中" + (found ? "包含" : "不包含") + target);
        sc.close();
    }
}
```